



Széchenyi István Katolikus Technikum és Gimnázium

Szoftverfejlesztő és -tesztelő projektfeladat

„VÍZCSEPP MOSÁS”

Készítették:

Galambosi Dániel,

Megyery Bese Máté,

Kiss Bálint

Ózd, 2026

Bevezetés.....	3
1. A webalkalmazás működéséhez szükséges szoftverek	4
2. A webalkalmazás indítása	5
2.1. Backend indítása	5
2.2. Frontend indítása	5
3. Alkalmazott fejlesztői és csoportmunka eszközök	6
3.1. Tervezés és projektmenedzsment	6
3.2. Fejlesztői környezet.....	6
3.3. Verziókezelés	6
3.4. Frontend technológiák	7
3.5. Backend technológiák	7
3.6. Szerver környezet	7
5. Frontend	9
6. Backend	10
7.Rendszerkövetelmények	11
7.1. Szerveroldali követelmények.....	11
7.2. Kliensoldali követelmények	11
7.3. Minimális hardverigény.....	11
8. Adatbázis dokumentáció.....	12
8.1. Áttekintés	12
8.2. Adatbázis szerkezete	12
8.3. Táblák részletes leírása.....	13
8.4. E-K Diagram	16
9. Backend dokumentáció	17
9.1. Fontos metódusok	17
9.2. Validálások és ellenőrzések	17
9.3.Visszatérési értékek	17
9.4. HTTP státuszkódok	17
9.5. Metódusok	18
9.6. Backend metódusok tesztjei:.....	22
10. Frontend dokumentáció.....	32
10.1. Oldalak bemutatása	34
10.2. Angular működés a projektben	41
10.3. Komponensek bemutatása	41
10.4. Frontend tesztelés.....	43
11. Továbbifejlesztési lehetőségek.....	46
Irodalomjegyzék	47

Bevezetés

A feladatunk egy modern, jól strukturált és felhasználóbarát webes alkalmazás felépítése volt egészen az alapoktól. Valami innovatívot kell bemutatnunk, ami segíti az embereket mindennapi életükben. Valamit, ami egy valós problémára nyújt megoldást. Egy fikcionális, autótakarítással foglalkozó vállalkozás számára készítettük, mivel nem mindenkinek van ideje vagy energiája igényesen és részletesen tisztán tartani az autóját/autóit.

A fejlesztés során a kliensoldali működéshez az Angular keretrendszert alkalmaztuk, amely lehetővé teszi a dinamikus, komponens-alapú felépítést és a reszponzív felhasználói felület kialakítását. A szerveroldali logikát PHP nyelven, Laravel keretrendszer segítségével valósítottuk meg, amely biztosítja az adatkezelést, az üzleti logikát és az API végpontokat. A fejlesztői környezet biztosításához XAMPP-ot használtunk, Apache webservert és MySQL adatbázis-kezelővel.

https://github.com/B-e-s-e/Vizsgaremek_2026

1. A webalkalmazás működéséhez szükséges szoftverek

A webalkalmazás megfelelő működéséhez és futtatásához több szoftver telepítése szükséges. Ezek biztosítják a backend és frontend környezet működését, valamint a fejlesztési folyamat támogatását.

A szükséges szoftverek:

- **XAMPP (Apache + MySQL)** – a helyi webszerver és adatbázis futtatásához
 - **PHP** – a backend működéséhez (a XAMPP részeként települ)
 - **Composer** – a Laravel backend függőségeinek telepítéséhez
 - **Node.js és npm** – az Angular frontend futtatásához
 - **Angular CLI** – az Angular alkalmazás fejlesztéséhez és futtatásához
 - **Visual Studio Code** – a projekt szerkesztéséhez
 - **Böngésző (Chrome, Edge, Firefox)** – az alkalmazás megjelenítéséhez
- Ezek a szoftverek együtt biztosítják a teljes fejlesztési és futtatási környezetet.

2. A webalkalmazás indítása

2.1. Backend indítása

A backend indításához az alábbi lépések szükségesek:

1. Indítsa el a XAMPP Control Panelt

2. Indítsa el az Apache és MySQL szolgáltatásokat

3. Nyissa meg a backend projekt mappáját Visual Studio Code-ban

4. Terminálban futtassa az alábbi parancsokat:

```
php artisan migrate --seed
```

```
php artisan serve
```

A backend ezek után az alábbi címen lesz elérhető:

2.2. Frontend indítása

A frontend futtatásához:

1. Nyissa meg a frontend projekt mappáját

2. Terminálban futtassa:

```
npm install
```

```
npm install bootstrap@5.3.8
```

```
ng serve
```

3. Alkalmazott fejlesztői és csoportmunka eszközök

A projekt során több különböző eszközt használtunk a hatékony fejlesztés és együttműködés érdekében.

3.1. Tervezés és projektmenedzsment

Figma

A felhasználói felület megtervezéséhez a Figma online tervezőprogramot használtuk. Segítségével előre elkészítettük a design terveket, így a fejlesztés során pontosan tudtuk követni a kívánt megjelenést.

Trello

A feladatok nyomon követésére és szervezésére a Trello alkalmazást használtuk. A feladatokat listákba és kártyákba rendeztük, így a csapat minden tagja átláthatta a munkafolyamatokat.

3.2. Fejlesztői környezet

Visual Studio Code (v1.96 vagy újabb)

A fejlesztés során a Visual Studio Code szerkesztőt használtuk, amely támogatja a HTML, CSS, JavaScript, Angular és PHP nyelveket. Kiegészítő bővítményekkel gyorsabb és hatékonyabb fejlesztést tett lehetővé.

3.3. Verziókezelés

GitHub

A projekt verziókezelésére a GitHub platformot használtuk. Segítségével a forráskód változásai követhetők voltak, valamint lehetővé tette a csapat tagjai közötti együttműködést.

3.4. Frontend technológiák

Angular (v17 vagy újabb)

A webalkalmazás kliensoldali részének fejlesztéséhez az Angular keretrendszert használtuk, amely komponens-alapú felépítést biztosít.

Node.js (v22 vagy újabb)

Az Angular alkalmazás futtatásához és a csomagkezeléshez Node.js környezetet használtunk.

3.5. Backend technológiák

Laravel (v10 vagy újabb)

A backend fejlesztéséhez a Laravel PHP keretrendszert használtuk, amely lehetővé teszi API k létrehozását és az adatbázis kezelését.

PHP (v8.1 vagy újabb)

A backend működéséhez szükséges programozási nyelv.

MySQL (v8 vagy újabb)

Az alkalmazás adatainak tárolására használt adatbázis-kezelő rendszer.

3.6. Szerver környezet

XAMPP (v8.1 vagy újabb)

A helyi fejlesztéshez használt környezet, amely tartalmazza az Apache webszervert, a MySQL adatbázist és a PHP futtatókörnyezetet.

4. Adatbázis

Az alkalmazás MySQL adatbázist használ az adatok tárolására.

A tárolt adatok:

- Felhasználók (név, email, telefonszám)
- Autók (márka, modell, évjárat, rendszám)

Az adatbázis kezelése:

- Laravel migrációk segítségével
- phpMyAdmin felületen keresztül

Ez biztosítja az adatok strukturált és biztonságos kezelését.

5. Frontend

A frontend Angular keretrendszerrel készült.

Feladata:

- felhasználói felület megjelenítése
- adatok bekérése (login, register)
- backend API hívása

Fő funkciók:

- bejelentkezés és regisztráció
- főoldal (szolgáltatások, árak)
- felhasználói fiók
- autók kezelése

6. Backend

A backend Laravel keretrendszerrel készült. Feladata:

- API biztosítása
- adatbázis kezelés
- felhasználók kezelése

Fő funkciók:

- regisztráció
- bejelentkezés
- autók kezelése
- megrendelések kezelése

7.Rendszerkövetelmények

7.1. Szerveroldali követelmények

- Operációs rendszer: Windows vagy Linux
- Backend: PHP (Laravel)
- Adatbázis: MySQL
- RAM: minimum 4 GB

7.2. Kliensoldali követelmények

- Böngésző: Chrome, Firefox, Edge
- Internetkapcsolat szükséges

7.3. Minimális hardverigény

- Legalább 2 magos processzor
- Minimum 4 GB RAM
- Stabil internetkapcsolat

8. Adatbázis dokumentáció

8.1. Áttekintés

Az adatbázis egy autómosó szolgáltatás ügyfél-, jármű-, munka- és szolgáltatáskezelő modulját valósítja meg. A rendszerben felhasználók rendelkeznek autókkal, a takarítók elvégzik a munkákat, és minden munkához hozzá tartozik egy szolgáltatás, amelynek ára rögzítésre kerül.

8.2. Adatbázis szerkezete

Az adatbázis öt táblából áll:

- 1. Felhasznalo – autótulajdonosok adatai**
- 2. Auto – felhasználókhoz tartozó járművek**
- 3. Takarito – munkát végző dolgozók**
- 4. Munka – elvégzett tisztítási munkák**
- 5. Szolgaltatas – munkák és árak összekötése(revízió előtt áll)**

Az adatbázis idegen kulcsokon keresztül kapcsolódik össze, biztosítva az adatok konzisztenciáját.

8.3. Táblák részletes leírása

8.3.1. Felhasznalo tábla

A felhasználókat (ügyfeleket) tárolja.

Mező	Típus	Leírás
ID	INT, PK, AUTO_INCREMENT	Egyedi felhasználói azonosító
nev	VARCHAR(100), NOT NULL	Felhasználó neve
phonenumber	VARCHAR(30)	Telefonszám
email	VARCHAR(100)	Email cím

Kapcsolatok:

- 1:N kapcsolat az **Auto** táblával (egy felhasználónak több autója lehet).
- 1:N kapcsolat a **Munka** táblával (egy felhasználó több munkát rendelhet).

8.3.2. Auto tábla

Az ügyfelekhez tartozó autókat rögzíti.

Mező	Típus	Leírás
ID	INT, PK, AUTO_INCREMENT	Egyedi autóazonosító
marka	VARCHAR(50)	Autó márkája
tipus	VARCHAR(50)	Autó típusa
evjarat	INT	Gyártási év
rendszam	VARCHAR(20), UNIQUE	Rendszám
tulaj	INT, FK → Felhasznalo(ID)	Az autó tulajdonosa

Kapcsolatok:

- N:1 a **Felhasznalo** táblával
- 1:N a **Munka** táblával

8.3.3. Takarito tábla

Az autómosó takarítóinak adatait tartalmazza.

Mező	Típus	Leírás
ID	INT, PK, AUTO_INCREMENT	Takarító azonosító
nev	VARCHAR(100), NOT NULL	Név
phonenumber	VARCHAR(30)	Telefonszám

Kapcsolatok:

- 1:N a **Munka** táblával (egy takarító több munkát végezhet)

8.3.4. Munkak tábla

A ténylegesen elvégzett munkákat tartalmazza. Minden munkához tartozik egy autó, egy felhasználó és opcionálisan egy takarító.

Mező	Típus	Leírás
ID	INT, PK, AUTO_INCREMENT	Munka azonosító
auto	INT, NOT NULL, FK → Auto(ID)	A tisztított autó
felhasznalo	INT, NOT NULL, FK → Felhasznalo(ID)	Megrendelő
takarito	INT, FK → Takarito(ID)	A munkát végző takarító
datum	DATE	Munka dátuma

Kapcsolatok:

- N:1 az **Auto** táblával
- N:1 a **Felhasznalo** táblával
- N:1 a **Takarito** táblával
- 1:N a **Szolgaltatas** táblával

8.3.5. Szolgaltatas tábla

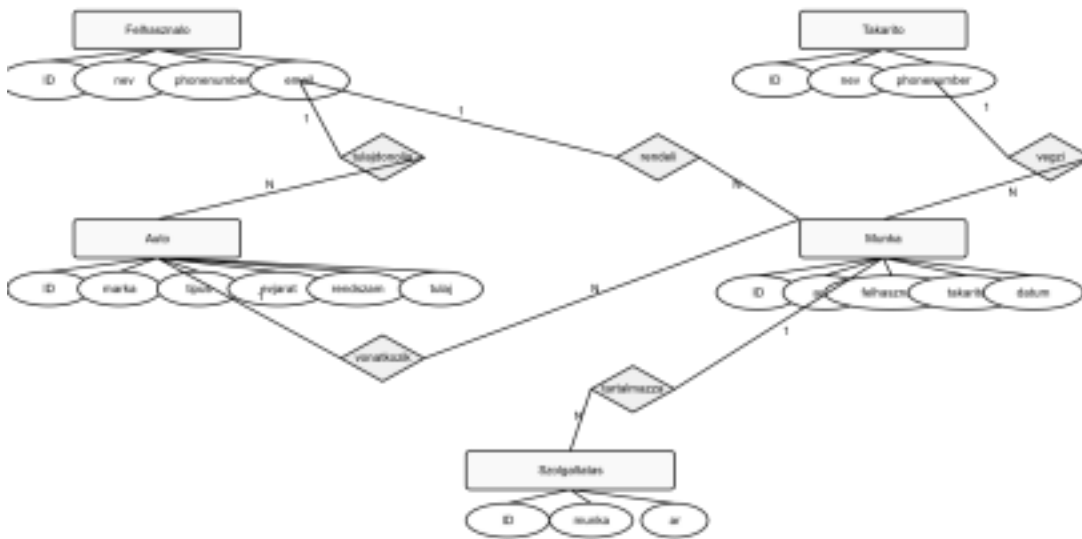
A munkához tartozó szolgáltatás(ok) árát rögzíti.

Mező	Típus	Leírás
ID	INT, PK, AUTO_INCREMENT	Szolgáltatás azonosító
munka	INT, NOT NULL, FK → Munka(ID)	Melyik munkához tartozik
ar	INT, NOT NULL	Szolgáltatás ára (HUF)

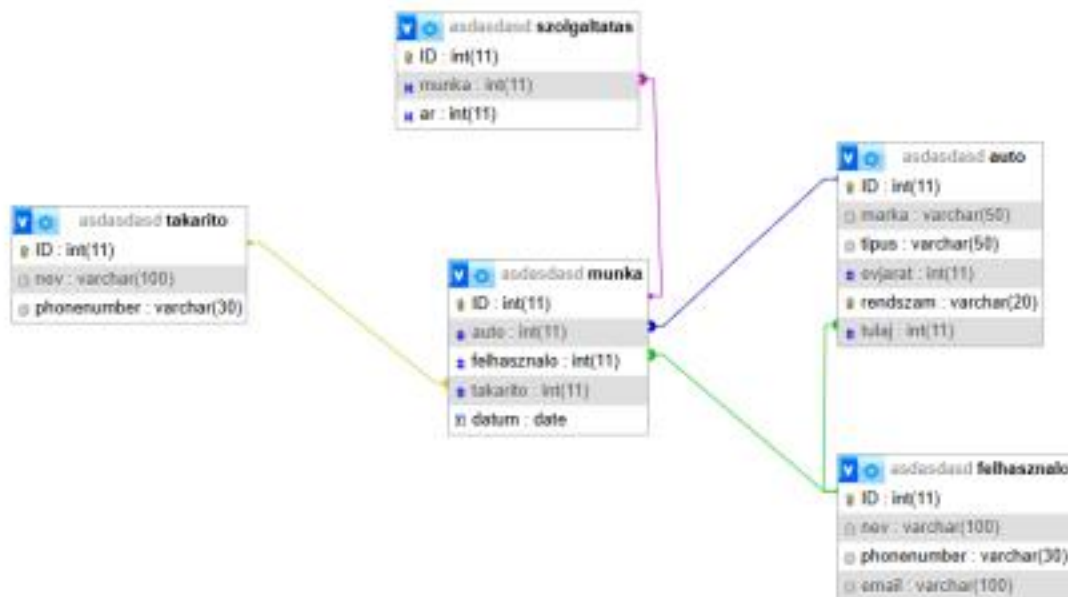
Kapcsolatok:

- N:1 a **Munka** táblával

8.4. E-K Diagram



1. E/K Diagram



2. Kapcsolatok

9. Backend dokumentáció

9.1. Fontos metódusok

- **store** – új autó létrehozása
- **update** – meglévő autó frissítése
- **getByOwner** – autók lekérdezése felhasználó alapján

9.2. Validálások és ellenőrzések

- Kötelező mezők meglétének ellenőrzése
- Autó létezésének ellenőrzése frissítés előtt

9.3. Visszatérési értékek

- JSON formátumú válaszok
- Sikeres és hibás esetek elkülönítése

9.4. HTTP státuszkódok

- **201** – sikeres létrehozás
- **206** – sikeres frissítés
- **404** – nem található erőforrás
- **406** – hibás vagy hiányos adatok

9.5. Metódusok

9.5.1. store() metódus

```
public function store(Request $request)
{
    $validator=Validator::make($request->all(),
    [
        'márka'=>'required',
        'típus'=>'required',
        'évjárat'=>'required',
        'rendszer' => 'required',
        'felhasználó' => 'required'
    ]);

    if($validator->fails())
    {
        return response()->json(['Hibüzenet'=>'Valamelyik kötelező adat hiányzik'],406);
    }

    $auto=Auto::create($request->all());
    return response()->json(['0] autó létrehozva. Rendszer:'=>$auto->rendszer],201);
}
```

3. store() metódus

A **store()** metódus új autó rögzítéséért felelős. A metódus egy HTTP kérésből (**Request**) kapja meg az adatokat, amelyek tartalmazzák az autó márkáját, típusát, évjáratát, rendszámát és a hozzá tartozó felhasználót. Ezek az adatok kötelezőek, ezért a metódus validációt végez annak ellenőrzésére, hogy minden szükséges mező kitöltésre került-e. Amennyiben valamelyik adat hiányzik, a rendszer egy JSON formátumú hibaválaszt ad vissza „Valamelyik kötelező adat hiányzik” üzenettel, 406-os (Nem elfogadható) HTTP státuszkóddal. Ha a validáció sikeres, az autó adatai elmentésre kerülnek az adatbázisba, és a rendszer 201-es (Létrehozva) státuszkóddal jelzi az erőforrás sikeres létrehozását, valamint visszaküldi az újonnan létrehozott autó rendszámát.

9.5.2. getByOwner() metódus

```
public function getByOwner(Request $request){
    $auto = Auto::where('felhasználó', '=', $request -> felhasználó);
    if($auto -> exists()){
        return $auto -> get();
    }
    else{
        return response() -> json(['Hiba:' => 'Nincs regisztrálva autó ezzel a rendszámmal.'], 404);
    }
}

public function update(Request $request, $id){
```

4. gtByOwner() metódus

A **getByOwner()** metódus egy adott felhasználóhoz tartozó autók lekérdezésére szolgál. A metódus a kérésben kapott felhasználónév alapján keres az adatbázisban. Először ellenőrzi, hogy létezik-e legalább egy autó, amely ehhez a felhasználóhoz van rendelve. Amennyiben található

ilyen bejegyzés, a rendszer visszaadja az autók listáját JSON formátumban. Ha nem található egyetlen autó sem, akkor a rendszer 404-es (Nem található) HTTP státuszkóddal tér vissza, és egy hibaüzenetet küld, amely jelzi, hogy nincs regisztrált autó az adott feltételekkel.

9.5.3. update() metódus

```
public Function update(Request $request, $id){
    $auto = Auto::find($id);
    if(is_null($auto)){
        return response()->json(['Hiba!' => 'Nem található a frissíteni kívánt autó.'], 404);
    }
    else{
        $validator=Validator::make($request->all(), [
            'marka'=>'required',
            'tipus'=>'required',
            'vejarat'=>'required',
            'rendszám' => 'required',
            'teljesitmeny' => 'required'
        ]);
        if($validator->fails()){
            return response()->json(['Hibás üzenet'=>'Valamelyik kötelező adat hiányzik'],406);
        }
        else{
            $auto->update($request);
            return response() -> json(['Autó sikeresen frissítve!' => $auto -> rendszám], 206);
        }
    }
}
```

5. update() metódus

Az **update()** metódus egy már meglévő autó adatainak frissítését végzi el az autó azonosítója alapján. A metódus első lépésként megvizsgálja, hogy az adott azonosítóval létezik-e autó az adatbázisban. Ha nem található ilyen rekord, a rendszer 404-es (Nem található) státuszkóddal és egy „Nem található a frissíteni kívánt autó” üzenettel válaszol. Amennyiben az autó létezik, a metódus ismét validálja a beérkező adatokat, mivel a frissítés során is minden mező kitöltése kötelező. Sikertelen validáció esetén a rendszer 406-os (Nem elfogadható) státuszkóddal jelez hibát. Ha minden adat megfelelő, az autó frissítésre kerül, és a rendszer 206-os (Részleges tartalom) HTTP státuszkóddal válaszol, amely jelzi a frissítés sikerességét, valamint visszaküldi az érintett autó rendszámát.

9.5.4. destroy() metódus

```
87
88 public function destroy($id){
89     $auto = Auto::find($id);
90     if(is_null($auto)){
91         return response('A törölni kívánt autó nem található.', 404);
92     }
93     else{
94         $auto -> delete();
95         return response('Autó eltávolítva az adatbázisból.', 202);
96     }
97 }
98
99
100
```

6. destroy() metódus

A **destroy(\$id)** metódus egy autó törlésére szolgál az adatbázisból egy megadott azonosító alapján. A metódus paraméterként megkapja az autó azonosítóját, majd megpróbálja megkeresni azt az adatbázisban. Ha az autó nem létezik, akkor egy 404-es (Nem található) HTTP státuszkóddal tér vissza, és azt az üzenetet adja vissza, hogy a törölni kívánt autó nem található. Amennyiben az autó létezik, a metódus törli azt az adatbázisból, majd 202-es (Elfogadva) státuszkóddal válaszol, „Autó eltávolítva az adatbázisból.” üzenettel. Ez a működés azért fontos, mert megakadályozza, hogy nem létező adatokat próbáljunk törölni, és egyértelmű visszajelzést ad a kliens számára.

9.5.5. searchByName() metódus

```
62  
63 public function searchByName(Request $request){  
64     $felhasznalo = Felhasznalo::where('nev', '=', $request -> nev);  
65     if($felhasznalo -> exists()){  
66         return $felhasznalo -> get();  
67     }  
68     else{  
69         return response() -> json(['Hiba:' => 'Nem található a keresésnek megfelelő felhasználó.'], 404);  
70     }  
71 }  
72
```

7. searchByName() metóduss

A **searchByName(Request \$request)** metódus felhasználók keresését végzi név alapján. A metódus egy HTTP requestet kap paraméterként, amely tartalmazza a keresett felhasználó nevét. A controller az adatbázisban megvizsgálja, hogy létezik-e olyan felhasználó, akinek a neve megegyezik a megadott értékkel. Ha van találat, akkor a metódus visszaadja a megtalált felhasználó vagy felhasználók adatait, alapértelmezett 200-as (OK) HTTP státuszkóddal. Ha nincs találat, akkor 404-es (Nem található) státuszkóddal tér vissza, és egy JSON formátumú hibaüzenetet küld, amely jelzi, hogy nem található a keresésnek megfelelő felhasználó. Ez a metódus azért fontos, mert lehetőséget ad az adatok gyors visszakeresésére, miközben a hibás vagy eredménytelen kereséseket is megfelelően kezeli.

9.6. Backend metódusok tesztjei:

9.6.1. index()

- **index()** a takarítók táblán:

The screenshot shows a REST client interface with a GET request to `http://127.0.0.1:8000/api/takaritok`. The response is a JSON array of 5 objects, each representing a cleaning record. The response status is 200 OK, size is 2.48 KB, and time is 243 ms.

```
1 {
2   {
3     "id": 1,
4     "nev": "Kírdi tiszta",
5     "phoneNumber": "06709999991"
6   },
7   {
8     "id": 2,
9     "nev": "Márga Csaba",
10    "phoneNumber": "06709999992"
11  },
12  {
13    "id": 3,
14    "nev": "Móni Zoltán",
15    "phoneNumber": "06709999993"
16  },
17  {
18    "id": 4,
19    "nev": "Kiss Leona",
20    "phoneNumber": "06709999994"
21  },
22  {
23    "id": 5,
24    "nev": "Róznér Berni",
25    "phoneNumber": "06709999995"
26  },
27 }
```

8. *index()* a takarítók táblán

- **index()** a szolgáltatások táblán:

The screenshot shows a REST client interface with a GET request to `http://127.0.0.1:8000/api/uzolg`. The response is a JSON array of 3 objects, each representing a service record. The response status is 200 OK, size is 1.9 KB, and time is 280 ms.

```
1 {
2   {
3     "id": 1,
4     "munka": {
5       "id": 1,
6       "auto": 1,
7       "felhasznalo": 1,
8       "takarito": 1,
9       "datum": "2016-01-04"
10    },
11    "ar": 15000
12  },
13  {
14    "id": 2,
15    "munka": {
16      "id": 2,
17      "auto": 2,
18      "felhasznalo": 2,
19      "takarito": 2,
20      "datum": "2016-01-07"
21    },
22    "ar": 18000
23  },
24  {
25    "id": 3,
26    "munka": {
27      "id": 3,
```

9. *index()* a szolgáltatások táblán

- **index()** a munkak táblán:

GET https://127.0.0.1:8000/api/munkak Send

Status: 200 OK Size: 12.54 KB Time: 194 ms

Response Headers Cookies Results Docs

```

1 {
2   {
3     "id": 1,
4     "auto": {
5       "id": 1,
6       "marka": "audi",
7       "típus": "ad",
8       "évjarat": 1994,
9       "rendszám": "ABLR-430",
10      "felhasználó": 1
11    },
12    "felhasznalo": {
13      "id": 1,
14      "nev": "Kovács István",
15      "phoneNumber": "0620110000",
16      "email": "istvan.kovacs@example.com"
17    },
18    "takarito": {
19      "id": 1,
20      "nev": "Bíró Tamás",
21      "phoneNumber": "0670000000"
22    },
23    "datum": "2020-01-04"
24  },
25  {
26    "id": 2,
27    "auto": {

```

JSON Content Format

```

1 {
2
3
4   "nev": ""
5 }

```

10. index() a munkak táblán

- **index()** a felhasználók táblán:

GET https://127.0.0.1:8000/api/felhasznalok Send

Status: 200 OK Size: 3.92 KB Time: 89 ms

Response Headers Cookies Results Docs

```

1 {
2   {
3     "id": 1,
4     "nev": "Kovács István",
5     "phoneNumber": "0620110000",
6     "email": "istvan.kovacs@example.com"
7   },
8   {
9     "id": 2,
10    "nev": "Szabó Mária",
11    "phoneNumber": "0620110002",
12    "email": "maria.szabo@example.com"
13  },
14  {
15    "id": 3,
16    "nev": "Varga János",
17    "phoneNumber": "0620110003",
18    "email": "janos.varga@example.com"
19  },
20  {
21    "id": 4,
22    "nev": "Horváth Kitti",
23    "phoneNumber": "0620110004",
24    "email": "kitti.horvath@example.com"
25  },
26  {
27    "id": 5,

```

JSON Content Format

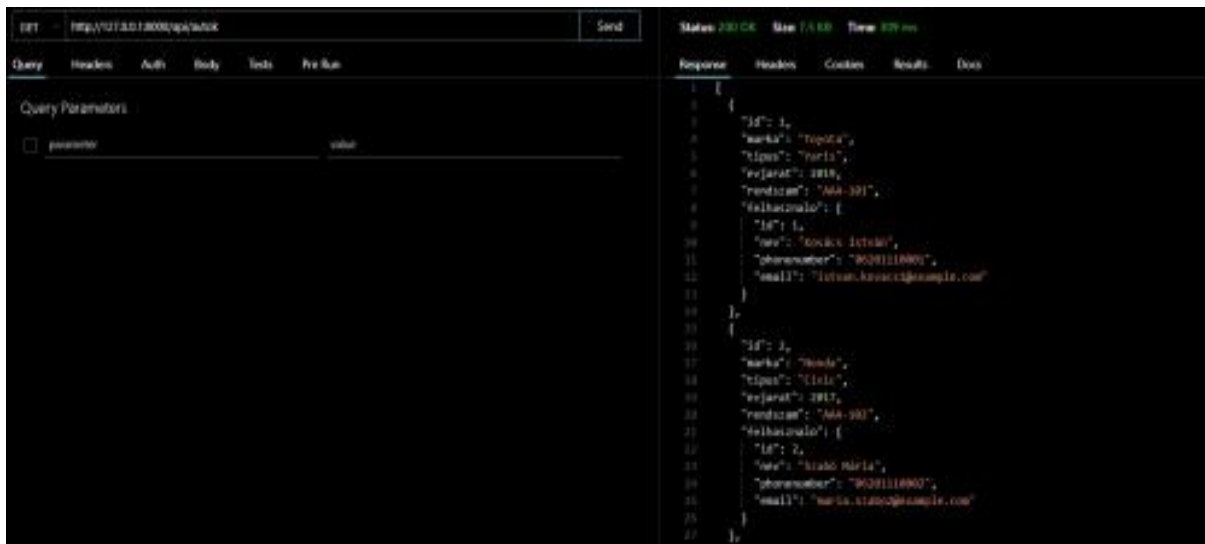
```

1 {
2
3
4   "nev": ""
5 }

```

11. index() a felhasználók táblán

- **index()** az autok táblán:



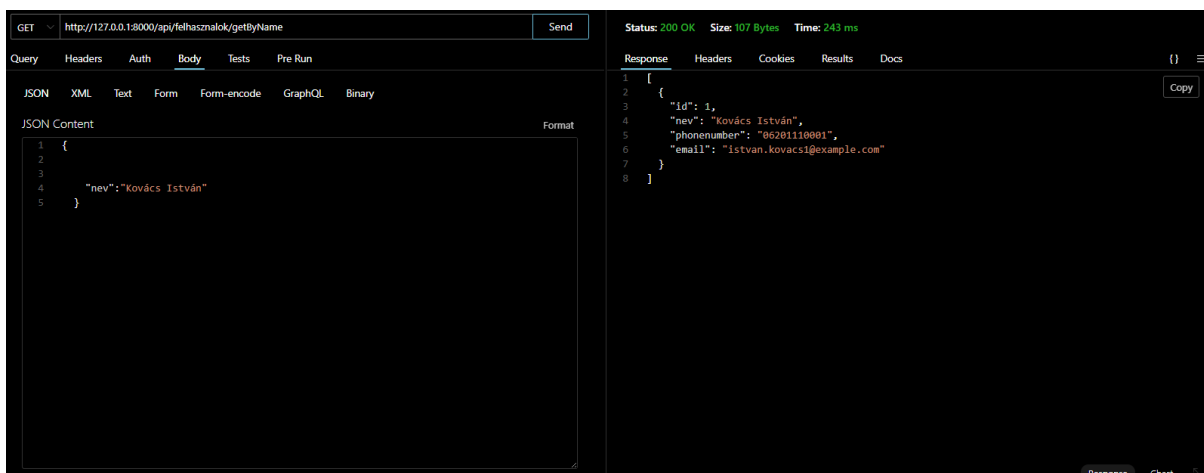
12. index() az autok táblán

9.6.2. getByNome()

A **getByNome()** metódus két táblán használható(**felhasznalok**, **takaritok**), és 2 lehetséges státuszkódot adhat vissza:

- Egy egyed sikeres lekérésénél visszaad egy **200**-as (OK) státuszkódot, illetve a lekért felhasználó adatait JSON formátumban.

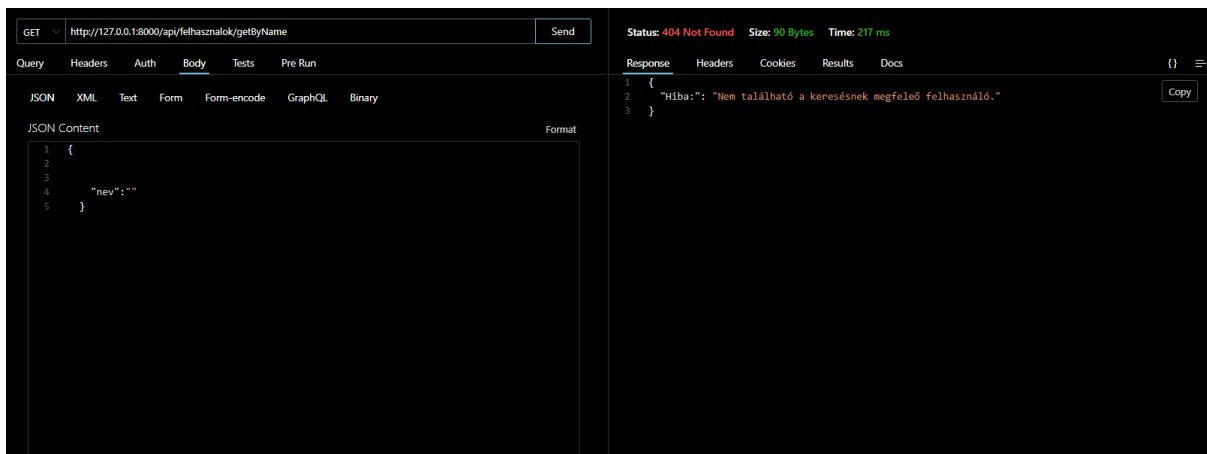
- Sikeres **getByNome()** a felhasználok táblán:



13. sikeres getByNome() a felhasználok táblán

- Sikertelen lekéréskor 404-es (Nem található) hibakódot ad vissza, illetve egy JSON formátumú üzenetet, miszerint nem található felhasználó a keresett névvel.

- Sikertelen **getByNome()** a felhasználok táblán:

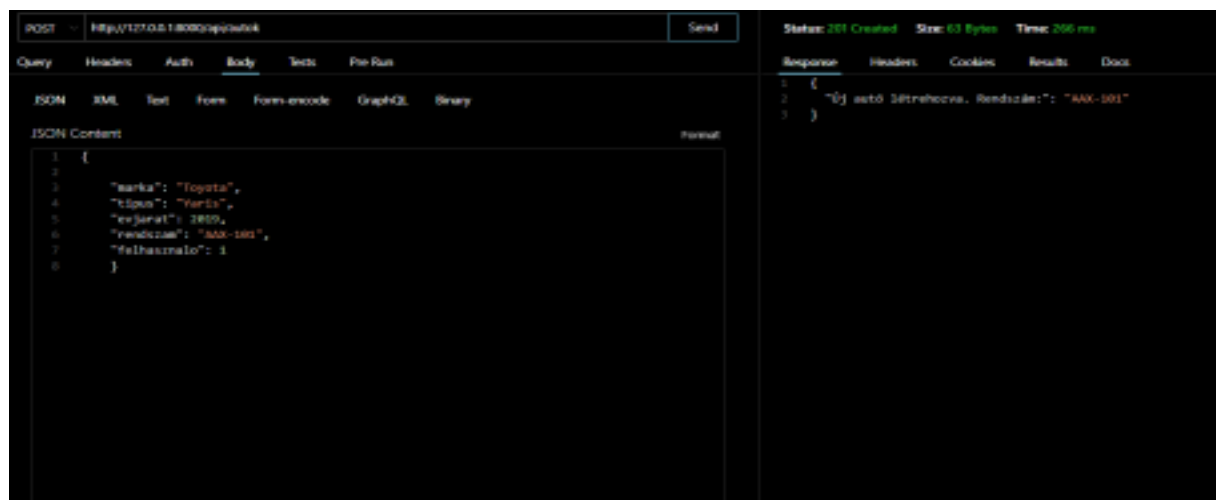


14. sikertelen `getByName()` a `felhasznalok` táblán

9.6.3. POST ()

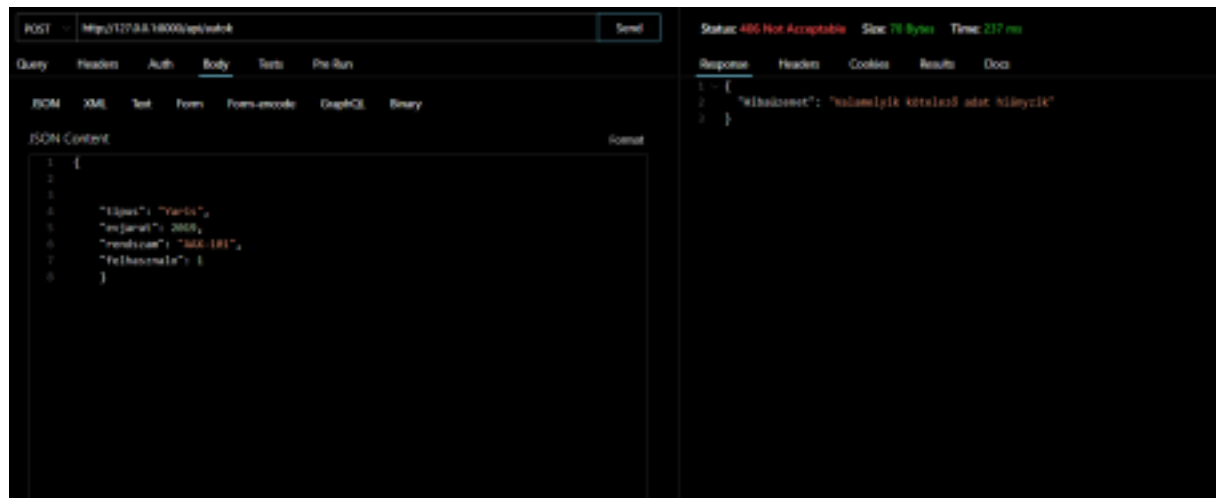
A **POST()** metódus 2 lehetséges státuszkódot adhat vissza:

- Egyed sikeres létrehozásakor **201**-es HTTP státuszkódot(létrehozva) ad vissza, illetve JSON formátumban egy létrehozást megerősítő üzenetet, ami a létrehozott autó rendszámát, felhasználó/takarító nevét, munka/szolgáltatás esetében dátumát tartalmazza.
- Sikeres **POST()** az **autok** táblán:



15. Sikeres `post()` az `autok` táblán

- Ha az egyed létrehozása sikertelen, a metódus **406**-os státuszkódot(nem elfogadható) ad vissza, illetve egy JSON formátumú üzenetet, miszerint a feltöltendő egyed adatformátuma hibás.
- Sikertelen **POST()** az **autok** táblán:

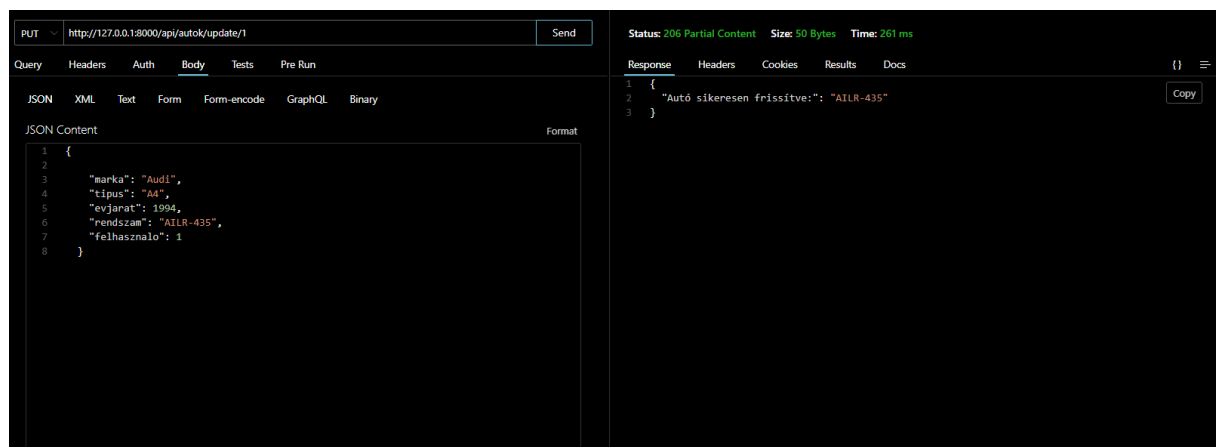


16. Sikertelen post() az autok táblán

9.6.4. update()

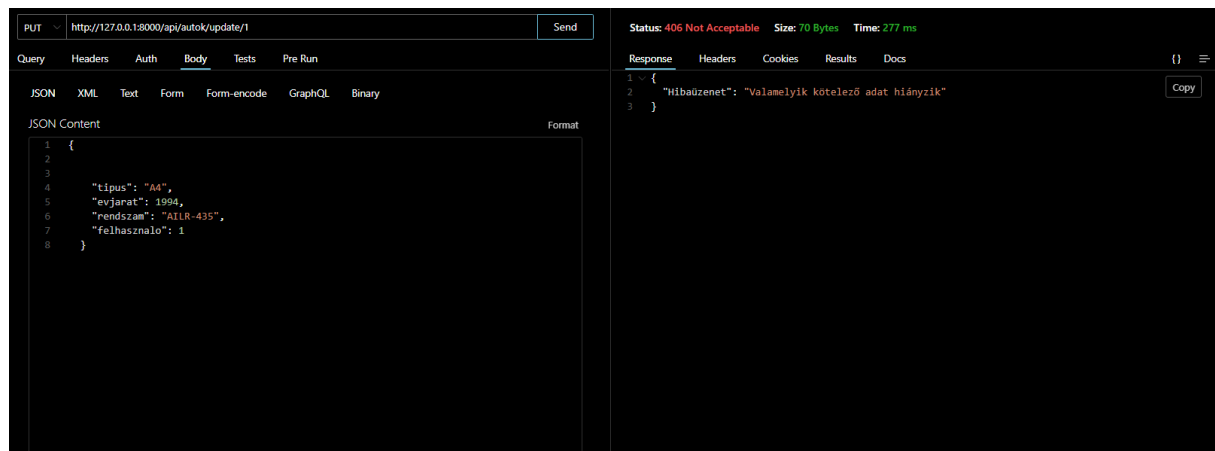
Az **update()** metódus 2 státuszkóddal térhet vissza:

- Egy egyed adatainak sikeres frissítésekor **206-os** (részleges tartalom) kóddal tér vissza.
- sikeres **update()** az **autok** táblán:



17. sikeres update() az autok táblán

- Sikertelen **update()** esetén **406-os** (Nem elfogadható) kódot küld vissza.
- sikertelen **update()** az **autok** táblán:

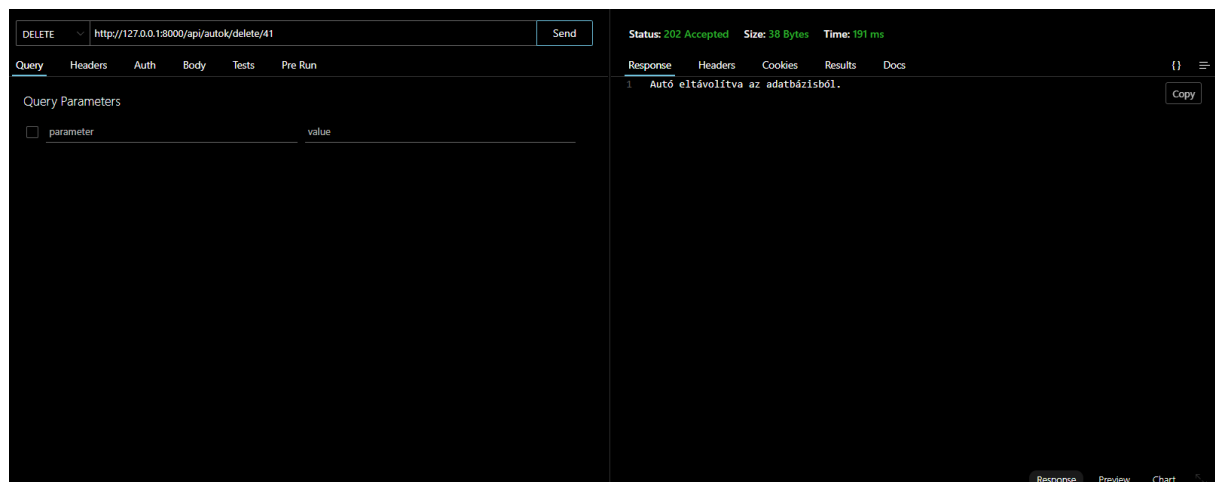


18. sikertelen update() az autok táblán

9.6.5. delete()

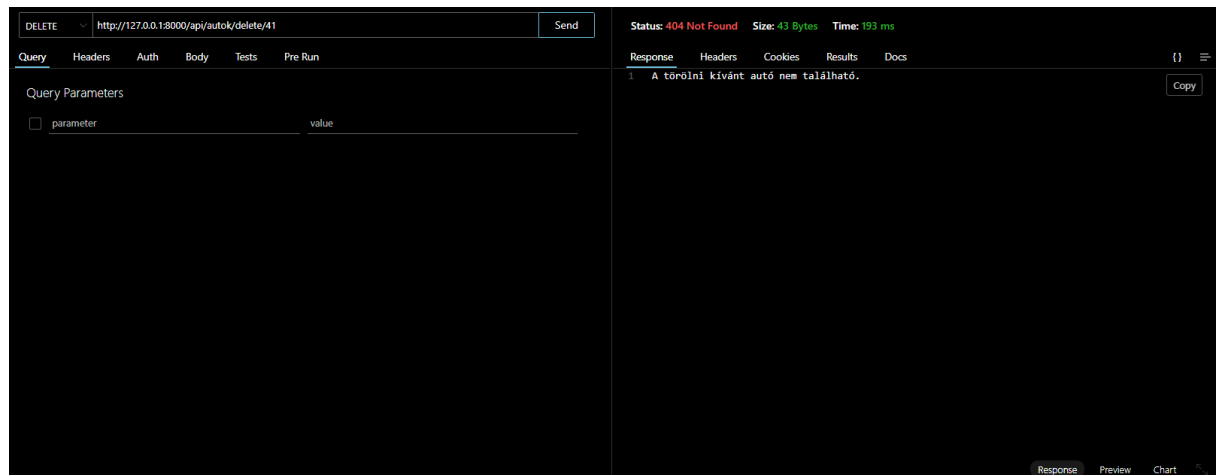
A **delete()** metódus 2 státuszkóddal térhet vissza:

- Egy egyed sikeres törlésekor **202-es** (elfogadva) kóddal.
- sikeres **update()** az **autok** táblán:



19. sikeres delete() az autok táblán

- Sikertelen **delete()** esetén **404**-es (nem található) kódot küld vissza.
- sikertelen **delete()** az **autok** táblán:

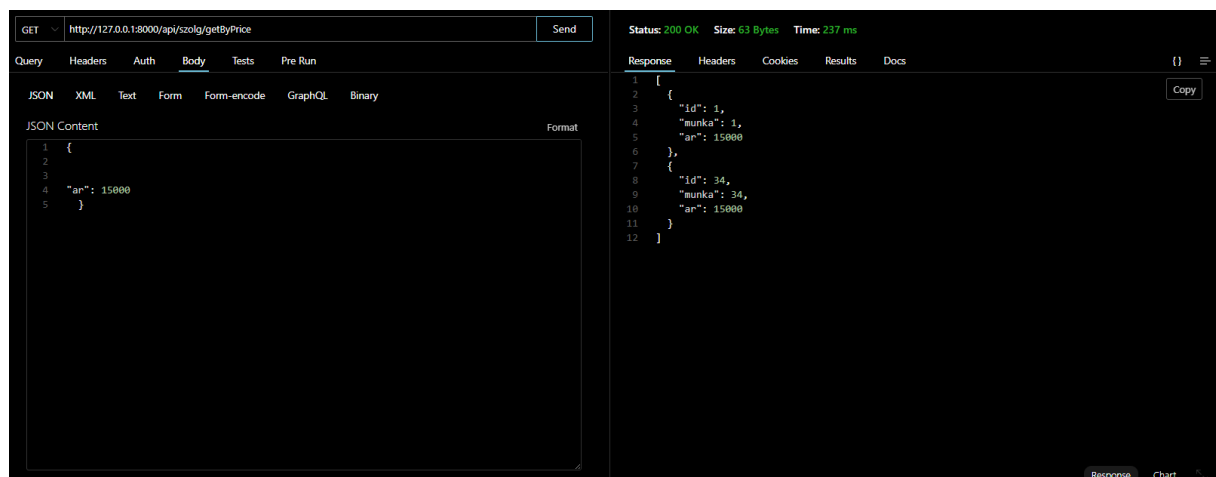


20. sikertelen `delete()` az autok táblán

9.6.6. `getByPrice()`

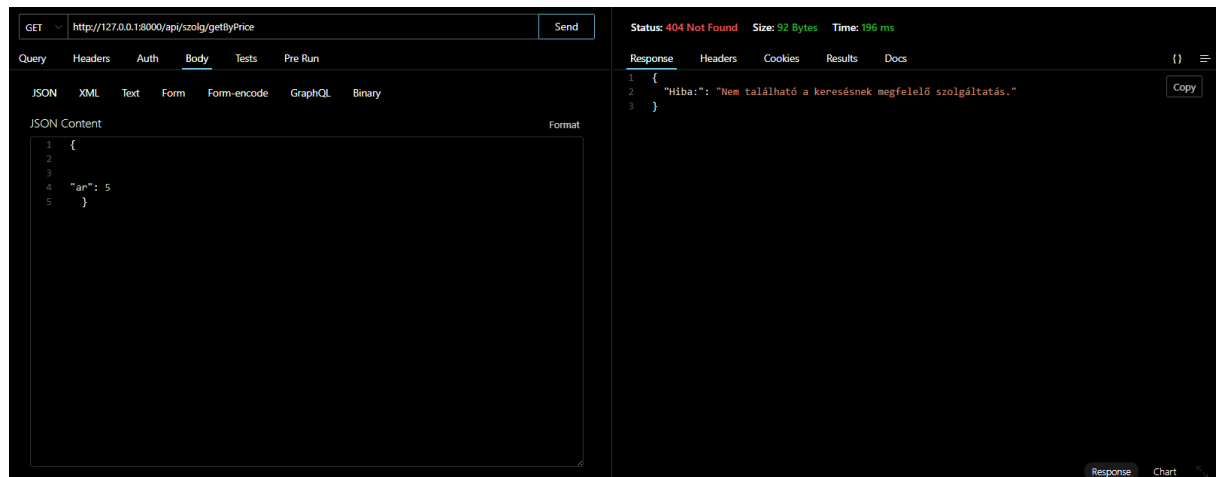
A **`getByPrice()`** metódus 2 státuszkóddal térhet vissza:

- Egy szolgáltatás sikeres lekérésekor **200**-as (OK) kóddal.
- sikeres **`getByPrice()`** a **szolgáltatások** táblán:



21. sikeres `getByPrice()` a szolgáltatások táblán

- Sikertelen **getByPrice()** esetén **404**-es (nem található) kódot küld vissza.
- sikertelen **getByPrice()** az **szolgáltatások** táblán:

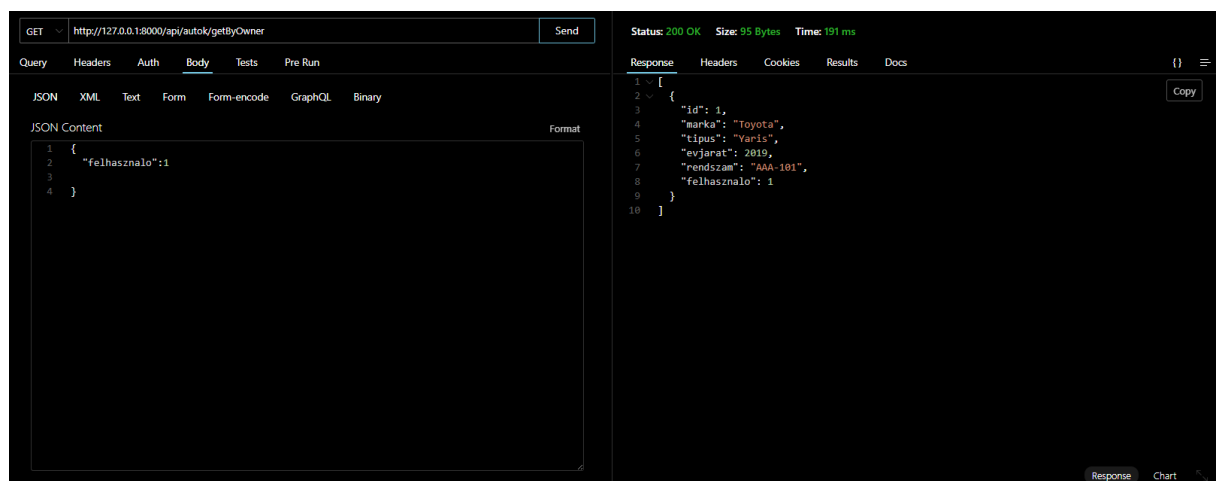


22. sikertelen **getByPrice()** a szolgáltatások táblán

9.6.7. **getByOwner()**

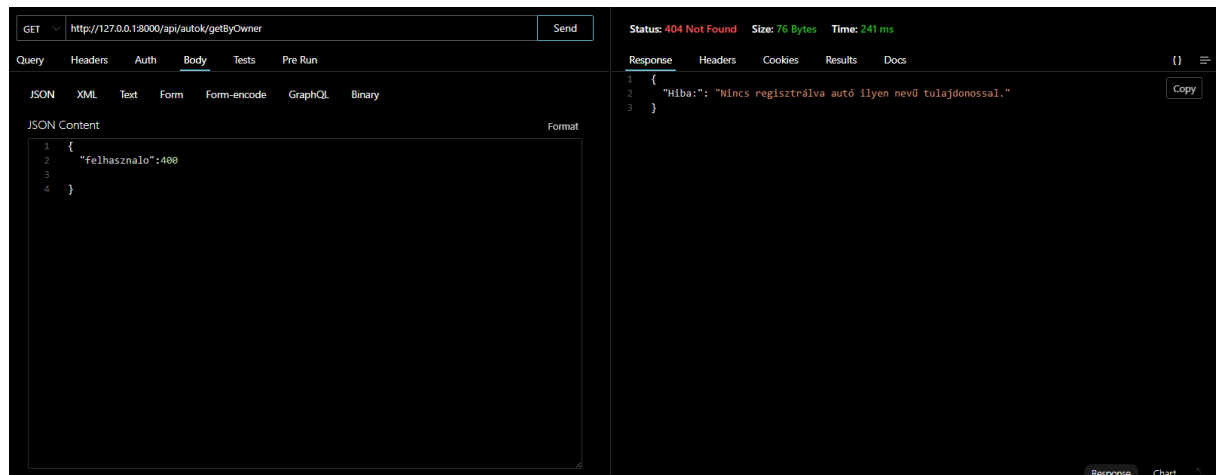
A **getByOwner()** metódus 2 státuszkóddal térhet vissza:

- Egy autó sikeres lekérésekor **200**-as (OK) kóddal.
- sikeres **getByOwner()** az **autok** táblán:



23. sikeres **getByOwner()** az autok táblán

- Sikertelen **getByOwner()** esetén **404**-es (nem található) kódot küld vissza.
- sikertelen **getByOwner()** az **autok** táblán:

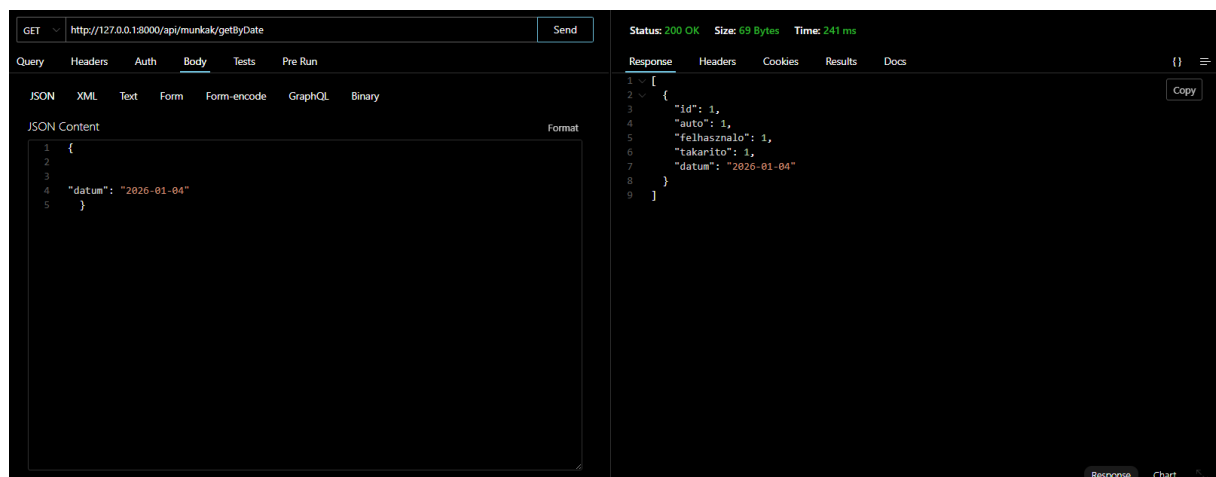


24. sikertelen **getByOwner()** az **autok** táblán

9.6.8. **getDate()**

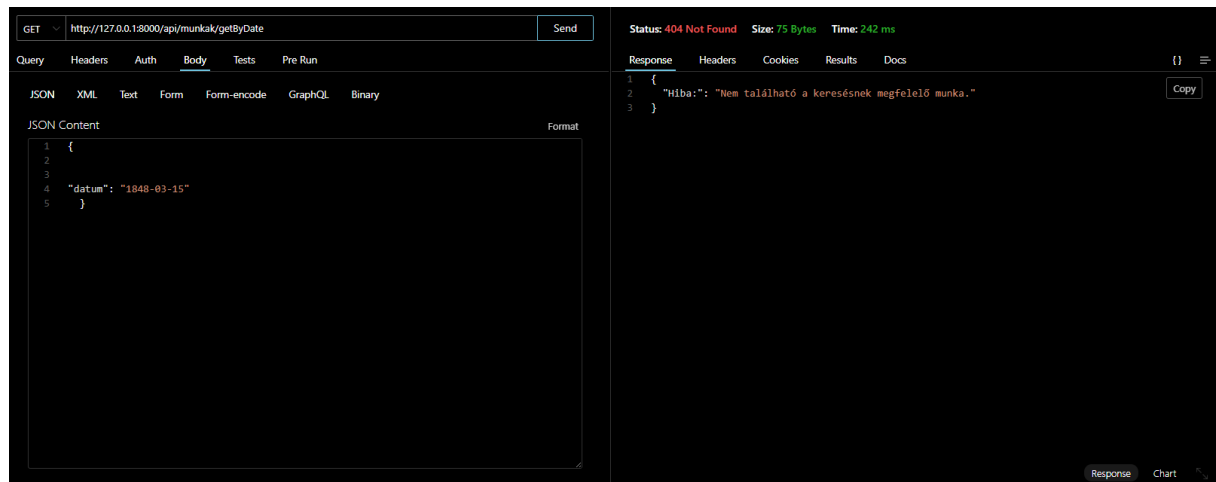
A **getDate()** metódus 2 státuszkóddal térhet vissza:

- Egy munka sikeres lekérésekor **200**-as (OK) kóddal.
- sikeres **getDate()** a **munkak** táblán:



25. sikeres **getDate()** a **munkak** táblán

- Sikertelen **getByDate()** esetén **404**-es (nem található) kódot küld vissza.
- sikertelen **getByDate()** a **munkak** táblán:



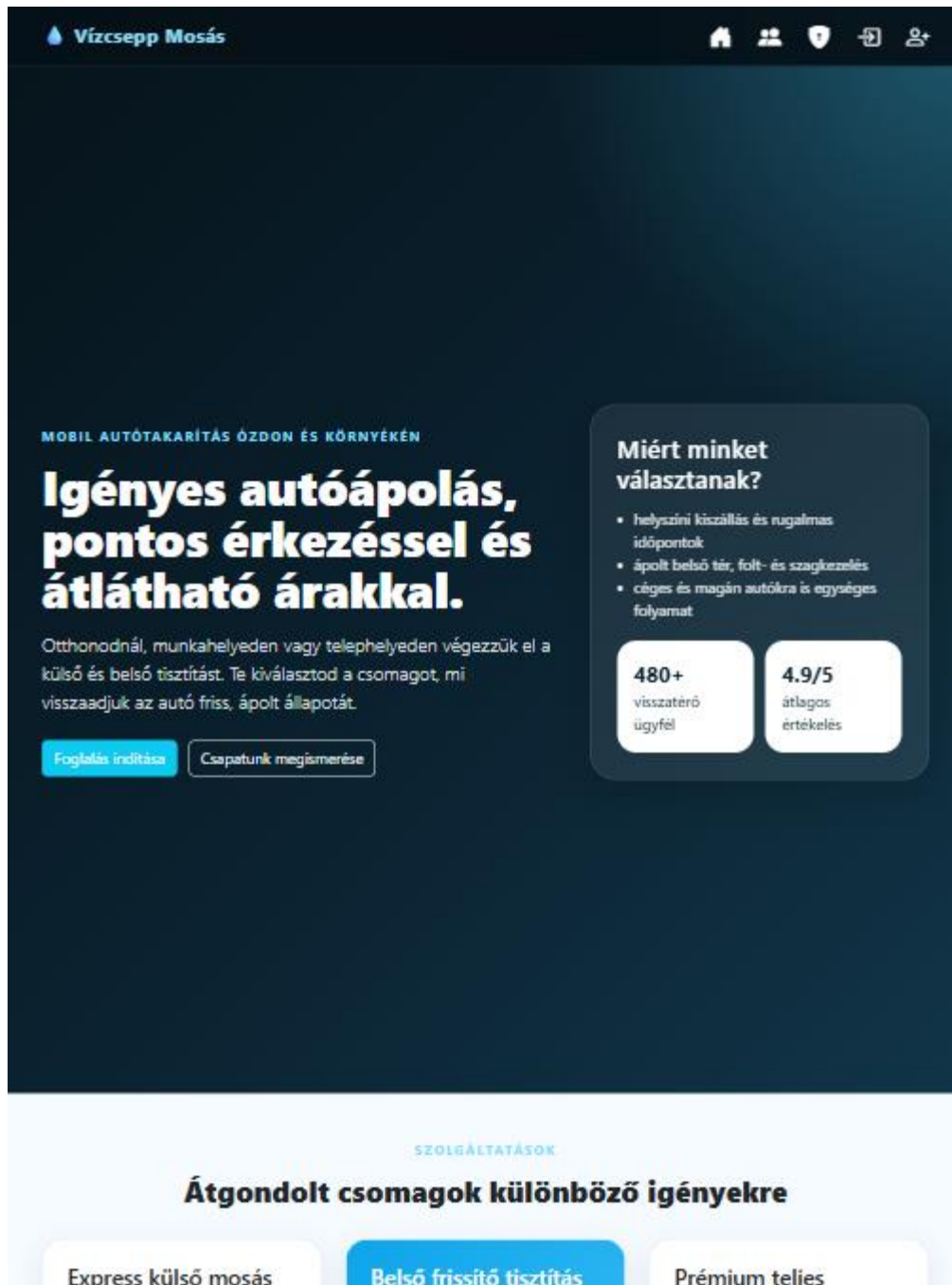
26. sikertelen `getByDate()` a munkak táblán

10. Frontend dokumentáció

Az egész webes alkalmazás reszponzív, az oldalak elemei a képernyő méretéhez és alakjához igazodnak. Ez Bootstrap 5.3.8-as keretrendszerrel van megoldva.



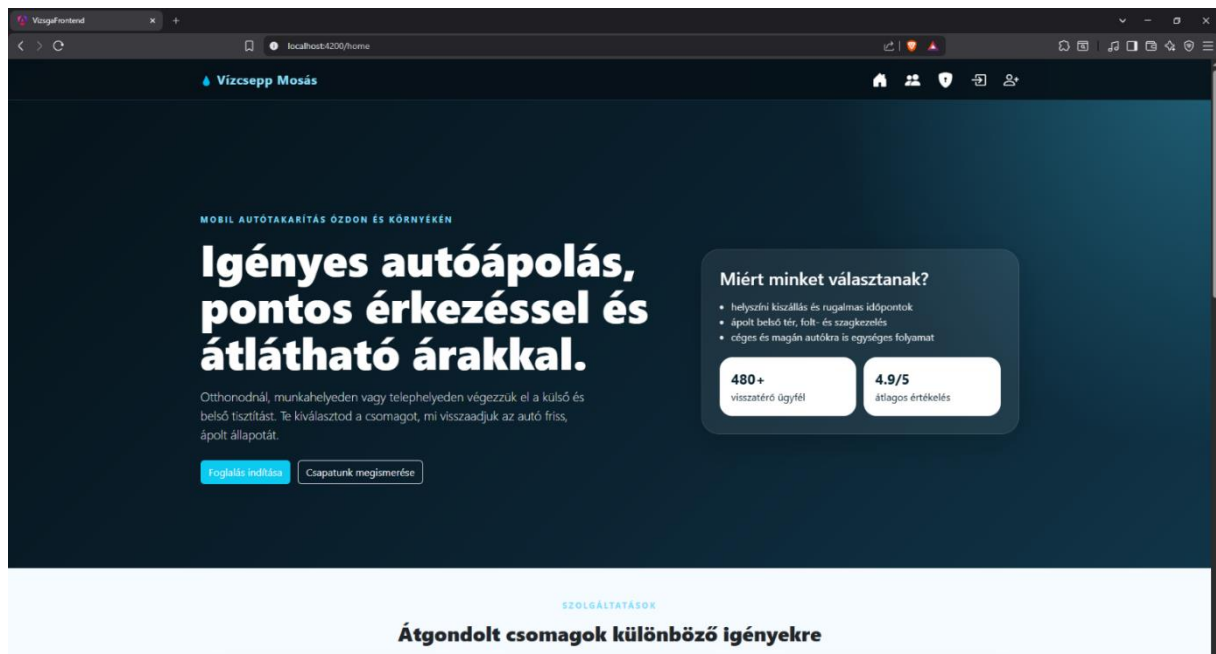
27. Főoldal mobil nézetben



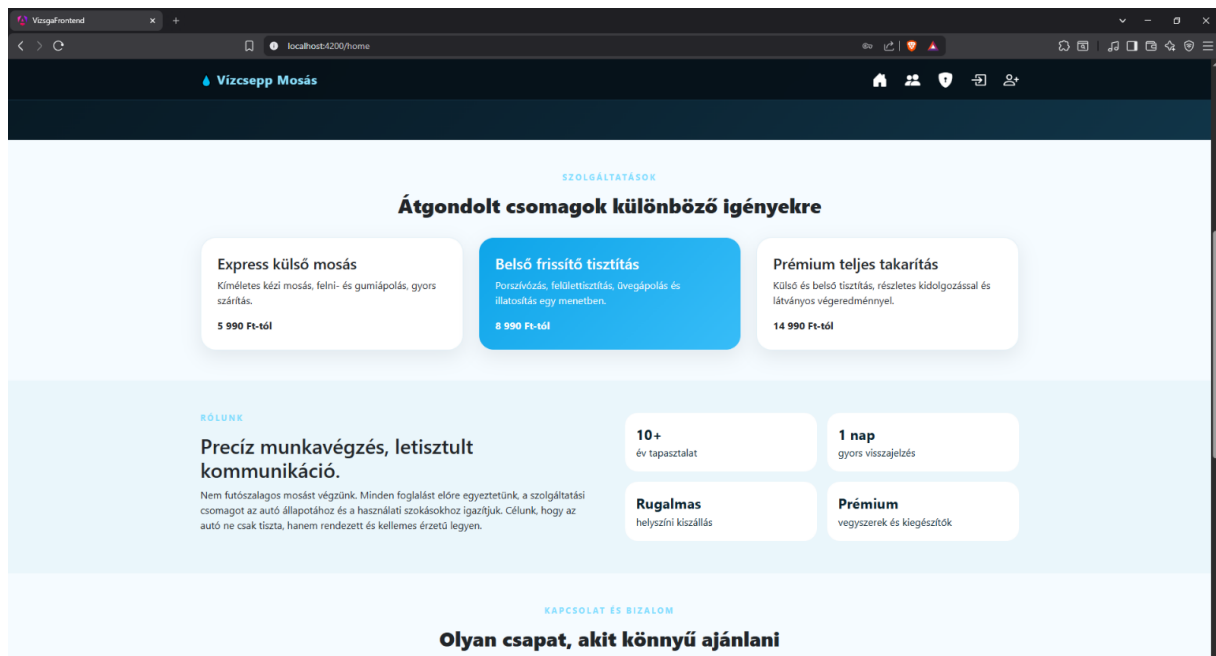
28. Főoldal tablet nézetben

10.1. Oldalak bemutatása

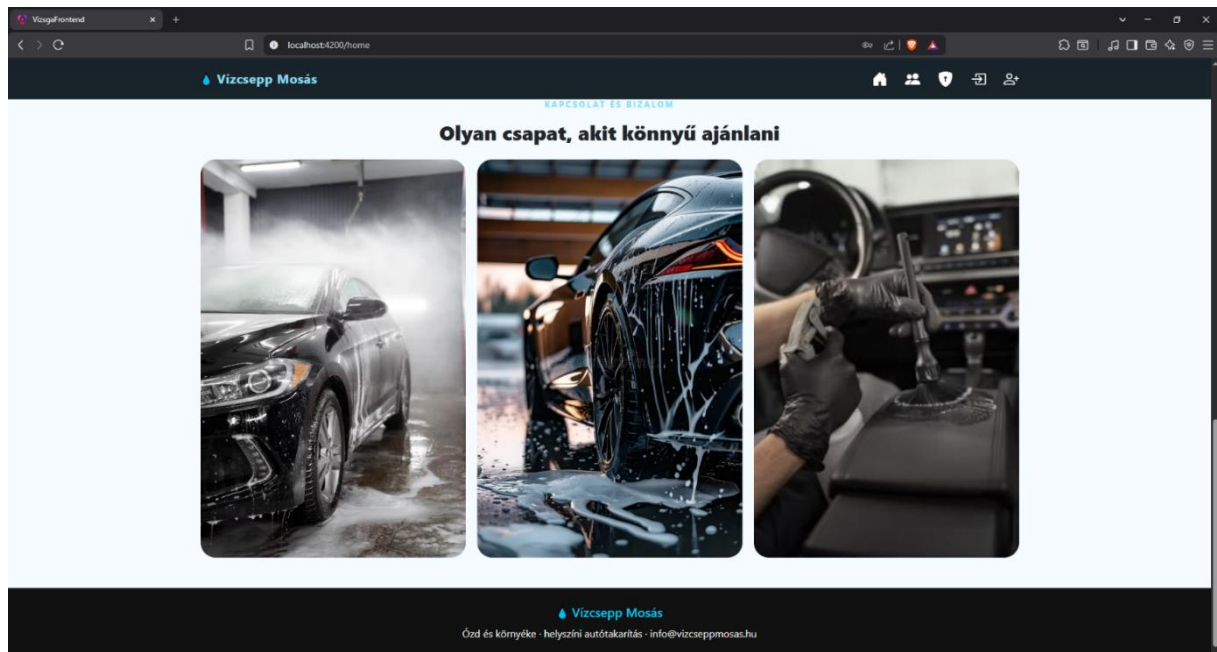
● Főoldal



29. Főoldal 1



30. Főoldal 2

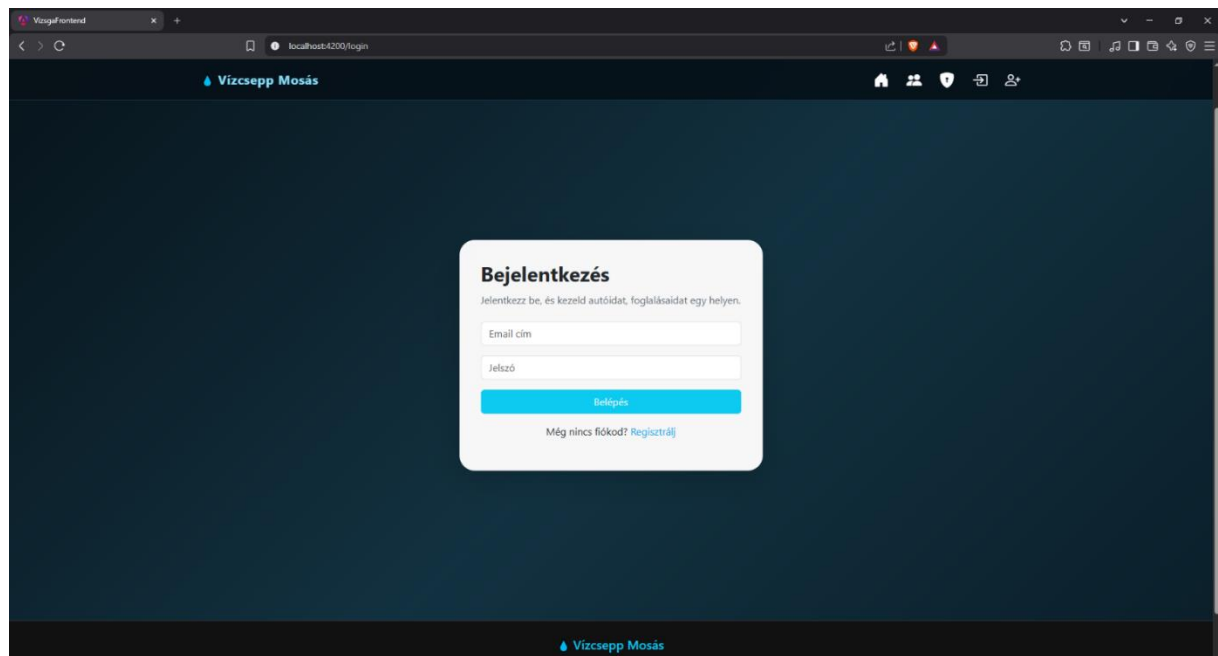


31. Főoldal 3

A főoldal a webalkalmazás kezdőfelülete, amely áttekintést nyújt a szolgáltatásokról és a vállalkozás működéséről. Itt található a legfontosabb információk, például az elérhető autómosási csomagok, árak, valamint a szolgáltatás előnyei.

A főoldal célja, hogy felkeltse a felhasználók érdeklődését, és egyszerű navigációt biztosítson a további funkciók felé. A felület reszponzív kialakítású, így különböző eszközökön (mobil, tablet, számítógép) is megfelelően jelenik meg.

- Bejelentkezés



32. Bejelentkezés oldal

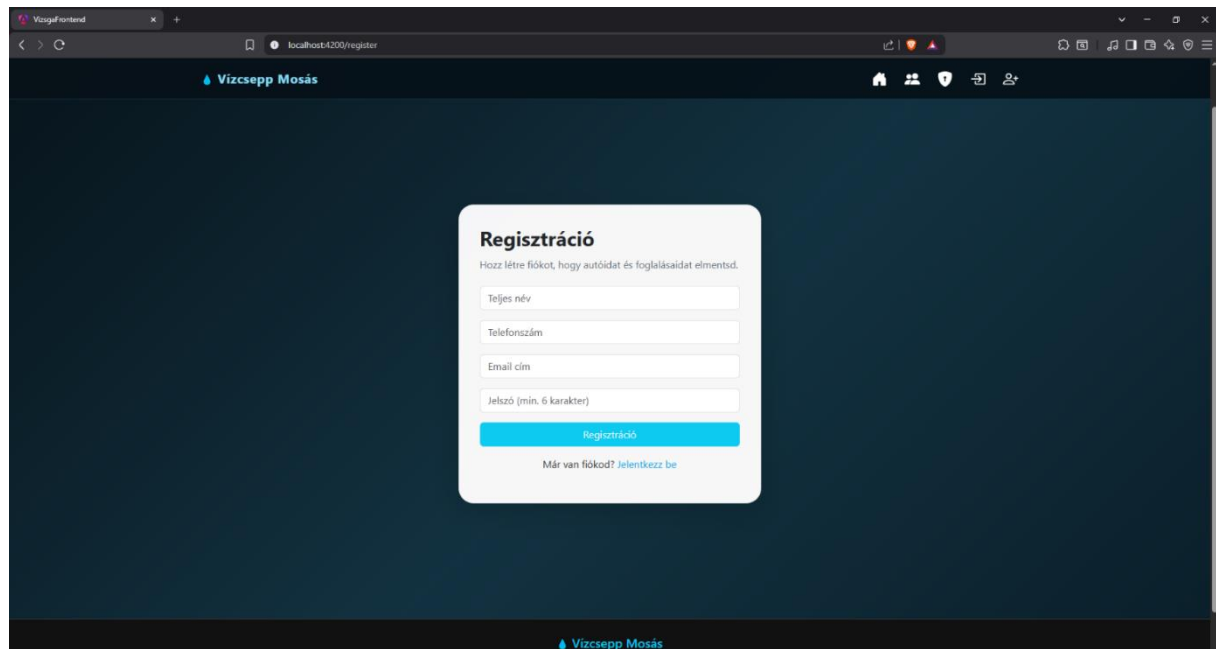
A bejelentkezési oldal a már regisztrált felhasználók számára készült. Itt az e-mail cím és jelszó megadásával lehet belépni a rendszerbe.

Sikeres bejelentkezés után a felhasználó átirányításra kerül a saját fiókjába. Hibás adatok megadása esetén a rendszer hibaüzenetet jelenít meg, és nem engedélyezi a belépést.

- Metódus:

```
login() {  
  this.errorMessage = '';  
  this.api.login({ email: this.inputEmail, password: this.inputPassword }).subscribe({  
    next: (res: any) => {  
      this.service.setSession(res.token, res.user);  
      this.router.navigate(['/account']);  
    },  
    error: (err) => this.errorMessage = err?.error?.message || 'Sikertelen bejelentkezés.'  
  });  
}
```

- Regisztráció



33. Regisztráció oldal

A regisztrációs oldal új felhasználók számára biztosít lehetőséget fiók létrehozására. A felhasználónak meg kell adnia a szükséges adatokat, például nevét, e-mail címét, telefonszámát és jelszavát.

Az űrlap validációt alkalmaz, így hibás vagy hiányos adatok esetén a rendszer nem engedi a regisztrációt. Sikeres regisztráció után a felhasználó automatikusan bejelentkezhet és hozzáférést kap a saját fiókjához.

- Metódus:
- `register() {`
- `this.errorMessage = '';`
- `this.api.register({`
- `nev: this.name,`
- `phonenumber: this.phonenumber,`
- `email: this.email,`
- `password: this.password`
- `}).subscribe({`
- `next: (res: any) => {`
- `this.service.setSession(res.token, res.user);`
- `this.router.navigate(['/account']);`
- `}`
- `}`

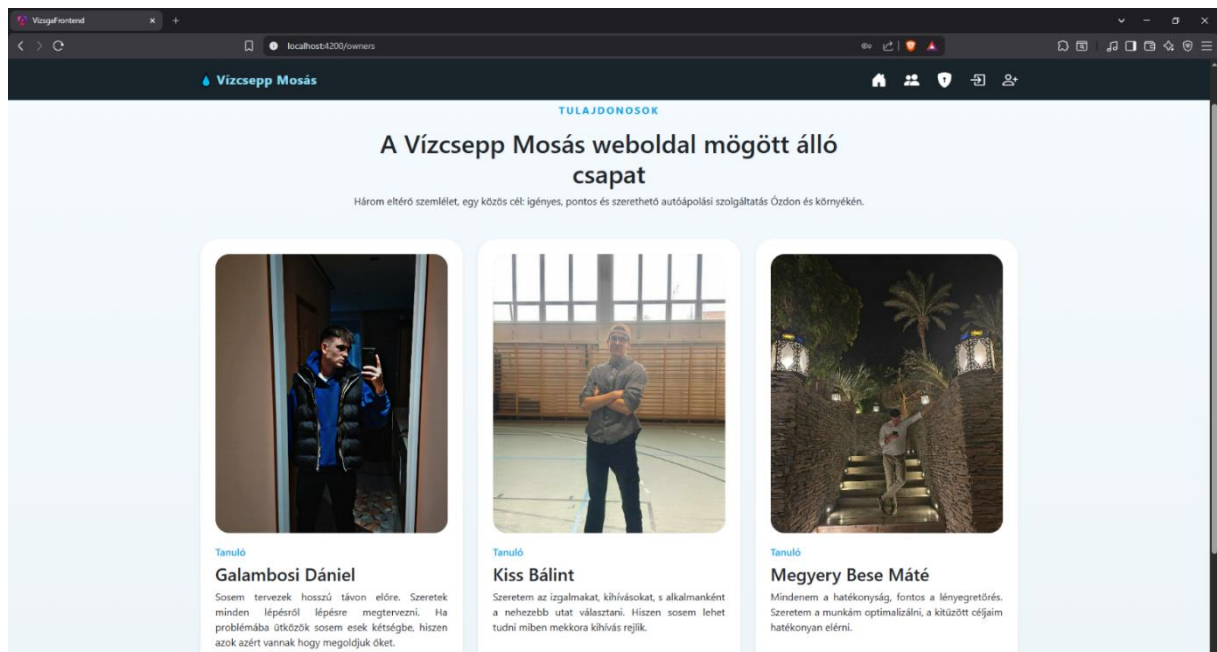
- ```
error: (err) => this.errorMessage = Object.values(err?.error?.errors || {}).flat().join(' ') || 'Sikertelen regisztráció.'
```
- ```
});
```
- ```
}
```

## • Felhasználói fiók

### 34. Felhasználói fiók

A felhasználói fiók regisztráció/bejelentkezés után elérhető. Ezen az oldalon tud egy felhasználó az adatbázisba felvinni tulajdonában álló autó(ka)t és onnan törölni azokat, változtathat a fiókadatain és igényelhet szolgáltatást.

## ●Tulajdonosok oldal

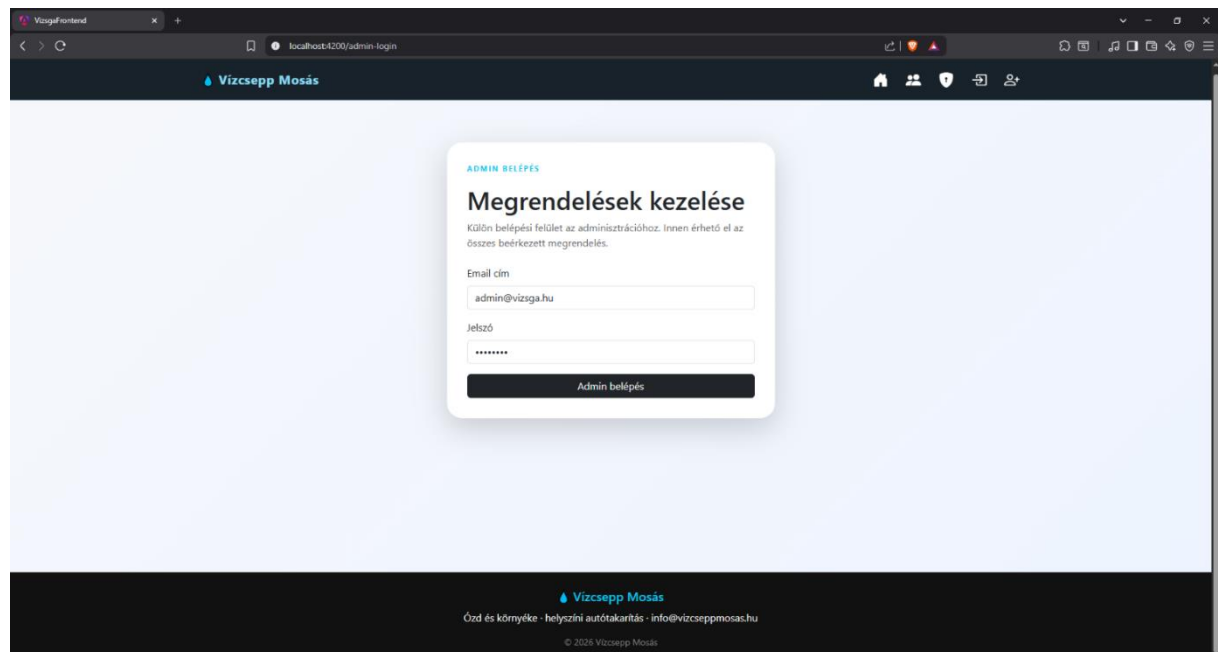


### 35. Tulajdonosok oldal

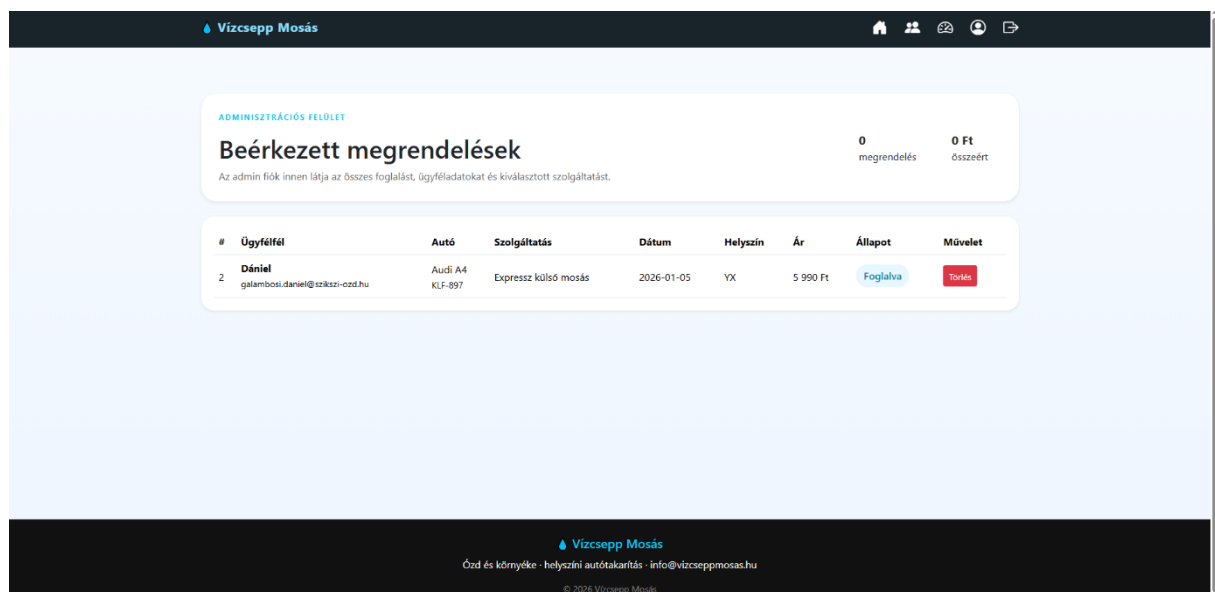
A tulajdonosok oldal az autótulajdonosok adatainak megjelenítésére és kezelésére szolgál. Itt a felhasználók áttekinthetik a rendszerben szereplő tulajdonosokat és a hozzájuk tartozó járműveket.

Ez az oldal segíti az adatok rendszerezését és gyors elérését, különösen adminisztratív feladatok esetén.

## • Admin felület



### 36. Admin bejelentkezési oldal



### 37. Admin felület

Az admin felület a rendszer adminisztrátorai számára készült, és teljes hozzáférést biztosít az alkalmazás adataihoz. Az adminisztrátor itt kezelheti a felhasználókat, autókat és megrendeléseket.

## 10.2. Angular működés a projektben

Az Angular alkalmazás moduláris felépítésű, amely komponensek segítségével kezeli az egyes funkciókat. A felhasználói felület és az adatok megjelenítéséért felel, HTTP kérésekkel kommunikál a backenddel.

Előnyök:

- újrafelhasználható kód
- gyors működés
- könnyű karbantartás

## 10.3. Komponensek bemutatása

- **Home**

A főoldal információkat nyújt a szolgáltatásról. Szerepelnek a vállalkozás előnyei, a kérhető takarítási csomagok és referenciaképek.

- **Login**

Bejelentkezési felület.

- **Register**

Új felhasználó létrehozása.

- **Account**

A felhasználói fiókot lehet rajta kezelni, szerkeszteni lehet az adatait, felhasználóhoz kötött autót felvenni az adatbázisba és törölni onnan, illetve magát a felhasználói fiókot törölni.

- **Admin**

Rendszergazdai oldal, a belépéshez külön admin fiók szükséges. Minden adathoz hozzáférés van.

Lehetőség van lefoglalt takarítások kilistázására, törlésére, vagy státuszának módosítására.

- megrendelések megtekintése

```
load() {
 this.loading = true;
 this.error = '';
 this.api.getAdminOrders().subscribe({ next: (res: any) => {
 this.orders = res;
 this.loading = false;
 }},
 error: (err) => {

 this.error = err?.error?.message || 'Nem sikerült lekérni a megrendeléseket.';
 this.loading = false;
 }
});
}
```

- törlés

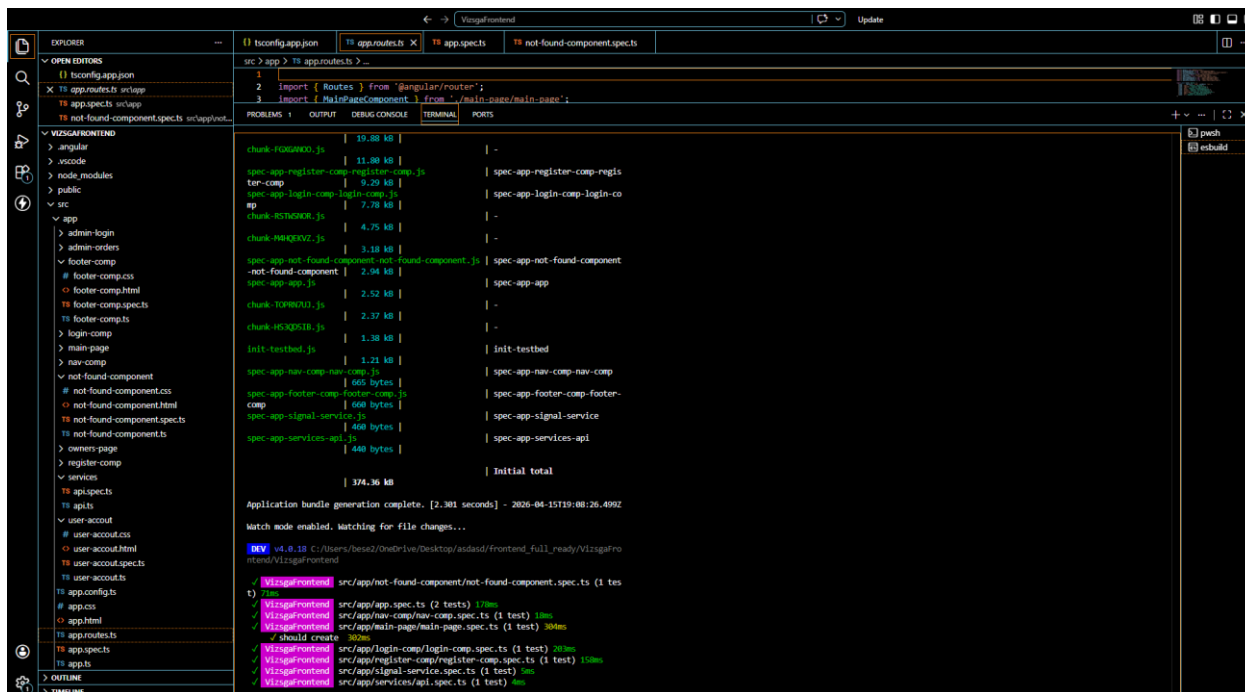
```
deleteOrder(id: number) {
 if (!confirm('Biztosan törölni szeretnéd ezt a megrendelést?')) return;

 this.api.deleteAdminOrder(id).subscribe({
 next: () => {
 this.info = 'A megrendelés törölve lett.';
 this.orders = this.orders.filter((o) => o.id !== id);
 },
 error: (err) => {
 this.error = err?.error?.message || 'Nem sikerült törölni a megrendelést.';
 }
 });
}
```

## 10.4. Frontend tesztelés

### 10.4.1. ng test

Az Angular CLI-ben lefuttatott teszt, ami segít megerősíteni, hogy a webes alkalmazás komponensei jó működnek-e.



38. Sikeres ng test

### 10.4.2. GUI tesztek

A bejelentkezési és regisztrációs felületeken beírt adatok a beküldés előtt automatikus ellenőrzésen mennek áthelyességük és meglétük ellenőrzése végett.

The screenshot shows a web application titled "Vizcsepp Mosás" with a dark blue header. The main content area is dark blue, and a white registration form is centered. The form is titled "Regisztráció" and includes the instruction "Hozz létre fiókot, hogy autódat és foglalásaidat elmentsd." It contains four input fields: a username field with "teszt", a phone number field with "123456789", an email field with "galambosidaniel@gmail.com", and a password field with "\*\*\*\*\*". Below the fields is a blue "Regisztráció" button. A red error message "Az e-mail cím már foglalt." is displayed below the button. At the bottom of the form, there is a link "Már van fiókod? Jelentkezz be".

39. GUI teszt – foglalt e-mail hiba

The screenshot shows the same web application as the previous one, but the registration form is now in a failed state. The input fields contain "12345", "12345", "12344", and "\*\*\*\*\*". The blue "Regisztráció" button is still present. A red error message "Sikertelen regisztráció." is displayed below the button. The link "Már van fiókod? Jelentkezz be" is still at the bottom of the form.

40. GUI teszt – túl rövid jelszó hiba

Vízcepp Mosás

### Regisztráció

Hozz létre fiókot, hogy autódát és foglalásaidat elmentsd.

12345

Telefonszám

Email cím

Jelszó (min. 6 karakter)

Regisztráció

A telefonszám mező kitöltése kötelező. Az e-mail mező kitöltése kötelező. A jelszó mező kitöltése kötelező.

Már van fiókod? [Jelentkezz be](#)

41. GUI teszt – hiányzó adat hiba

Regisztrálj'."/>

Vízcepp Mosás

### Bejelentkezés

Jelentkezz be, és kezeld autódát, foglalásaidat egy helyen.

123

Jelszó

Bejelentkezés

Sikertelen bejelentkezés.

Még nincs fiókod? [Regisztrálj](#)

42. GUI teszt – sikertelen bejelentkezés

## **11. Továbbfejlesztési lehetőségek**

Az alkalmazás fejlesztése közben későbbi továbbfejlesztésnek is hagyunk helyet. Lehetőség van egy automatizált visszajelző-rendszer kiépítésére, aminek segítségével foglaláskor a szolgáltatást igénylők e-mail-ben megerősítést kaphatnak.

Appon belüli online fizetési opcióra is van lehetőség, amivel a felhasználó a foglalás leadásakor előre ki tudná fizetni a szolgáltatást.

Egy mobilalkalmazást is lehetne fejleszteni ugyanezekkel a funkciókkal, így sokkal egyszerűbb és gyorsabb lenne a szolgáltatások igénylése.

## Irodalomjegyzék

- [1] Angular Documentation, <https://angular.io>
- [2] Laravel Documentation, <https://laravel.com>
- [3] PHP Documentation, <https://php.net>
- [4] MySQL Documentation, <https://dev.mysql.com>